

GDD TÉCNICO DE PROTOTIPO

Fórmulas, Módulos y Sistemas para un RPG Táctico

Arquitectura Anti-Gacha Orientada a Preparación Consciente

Documento derivado del GDD de Arquitectura Base. Especifica las fórmulas de combate y progresión, la estructura de datos, los módulos de software y el alcance recomendado para construir un prototipo jugable (vertical slice).

Versión 1.0 — Documento de especificación técnica

Índice

1. Alcance del Prototipo (Vertical Slice)
2. Estructura de Datos Base (Esquemas)
3. Módulo de Combate — Fórmulas
4. Módulo de Progresión de Personaje — Fórmulas
5. Módulo de Árbol de Talentos (Red por Bloques)
6. Módulo de Diagnóstico de Derrota (Motor de Causalidad)
7. Módulo de Expediciones y Energía
8. Módulo de Reclutamiento Táctico
9. Módulo de Economía y Monetización
10. Arquitectura de Software Recomendada
11. Plan de Construcción del Prototipo (Sprints)
12. Métricas de Balance a Vigilar

1. Alcance del Prototipo (Vertical Slice)

Antes de construir todos los sistemas a escala completa, se recomienda un prototipo vertical que demuestre el bucle central completo con el mínimo contenido posible. El objetivo no es impresionar visualmente, sino validar que el ciclo Desafío → Diagnóstico → Preparación → Reintento es divertido y comprensible.

1.1 Contenido mínimo necesario

Elemento	Cantidad	Detalle
Héroes jugables	3	Uno por vértice del Triángulo Táctico: Pícaro, Guerrero, Tanque
Habilidades Únicas	3	Una por héroe (ej. Asesino Silente, Muralla Inquebrantable, Ruptura de Guardia)
Nodos de talento por héroe	8-10	2-3 Núcleo, 2-3 Respuesta, 2 Sincronización, 2 Modificadores
Misión Principal (Jefe)	1	Debe tener al menos 2 mecánicas: una de daño sostenido (sangrado) y una de ruptura de escudo
Mazmorras secundarias	2	Una otorga material anti-sangrado; otra otorga material anti-escudo
Sistema de Diagnóstico	Versión mínima	Detecta 1 causa dominante de derrota por intento
Plantillas de build	2 por héroe	Sin costo de cambio fuera de combate

■ *Criterio de éxito del prototipo: un jugador que pierde contra el jefe debe ser capaz de, sin ayuda externa, leer el diagnóstico, ir a la mazmorra correcta, ajustar su árbol de talentos y ganar en el segundo o tercer intento.*

2. Estructura de Datos Base (Esquemas)

Estos esquemas son independientes del motor (Unity, Godot, etc.) y sirven como contrato de datos entre el diseño y la programación. Se recomienda implementarlos como archivos de configuración (JSON o ScriptableObjects) para que el balance pueda iterarse sin recompilar.

2.1 Esquema de Personaje

character.schema.json

```
{
  "id": "rogue_asesino_silente",
  "clase_base": "Rogue",
  "habilidad_unica": "asesino_silente",
  "rol_triangulo": "Picaro",          // Picaro | Guerrero | Tanque
  "stats_base": {
    "vida": 850, "ataque": 62, "defensa": 18,
    "velocidad": 108, "critico_pct": 15, "critico_dano_pct": 150
  },
  "crecimiento_por_nivel": {
    "vida": 24, "ataque": 2.1, "defensa": 0.6, "velocidad": 0.3
  },
  "nivel_actual": 1,
  "mastery": { "puntos": 0, "tope_actual": 20, "tope_hito": "capitulo_1" },
  "arbol_talentos": { "puntos_disponibles": 0, "nodos_desbloqueados": [] },
  "plantillas": [ { "nombre": "Objetivo Unico", "nodos": [] },
                  { "nombre": "Anti-Estados", "nodos": [] } ],
  "equipo": { "arma": null, "armadura": null, "accesorio": null }
}
```

2.2 Esquema de Nodo de Talento

talent_node.schema.json

```
{
  "id": "perforacion_osea",
  "rama": "Adaptacion",          // Especializacion | Adaptacion | Sincronizacion
  "tipo": "Respuesta",          // Nucleo | Respuesta | Sincronizacion | Modificador
  "costo_puntos": 2,
  "prerequisitos": ["nodo_inicial_asesino_silente"],
  "contrarresta_tags": ["armadura_alta", "daño_fisico_bloqueado"],
  "efecto": { "tipo": "conversion_dano", "origen": "fisico",
              "destino": "perforante", "porcentaje": 40 },
  "visible_desde_inicio": false // se revela tras derrota relevante
}
```

2.3 Esquema de Encuentro / Jefe

encounter.schema.json

```
{
  "id": "jefe_capitulo_1_verdugo",
  "vida": 42000,
  "fases": [
    { "umbral_vida_pct": 100, "mecanica": "sangrado_acumulativo",
      "tag_contador": "anti_sangrado" },
    { "umbral_vida_pct": 50, "mecanica": "escudo_ruptura",
      "tag_contador": "anti_escudo", "ventana_ruptura_seg": 6 }
  ],
  "modificador_ambiental": null // usado en variantes de endgame
}
```

3. Módulo de Combate — Fórmulas

El combate se resuelve por turnos simulados en tiempo real (velocidad determina orden de actuación), ejecutado automáticamente por la IA según reglas de prioridad, con puntos de intervención manual. Todas las fórmulas siguientes deben vivir en un módulo aislado (CombatResolver) que no dependa de la UI, para poder testear el balance por script.

3.1 Daño Base

Fórmula de Daño

```
DañoBruto = Ataque_Atacante * Multiplicador_Habilidad

FactorMitigacion = 100 / (100 + Defensa_Objetivo)

DañoMitigado = DañoBruto * FactorMitigacion

DañoFinal = DañoMitigado * ModTriangulo * ModCritico * ModAmbiental
```

Multiplicador_Habilidad es un valor propio de cada habilidad (ej. 1.4 para un ataque básico fuerte, 2.2 para una definitiva). ModAmbiental aplica los modificadores de mazmorra (ej. 0.5 si el encuentro reduce el daño de fuego a la mitad).

3.2 Triángulo Táctico

Atacante	Objetivo	Modificador de Daño	Modificador de Defensa recibida
Pícaro	Guerrero	+25%	—
Guerrero	Tanque	+25%	—
Tanque	Pícaro	+25%	—
Guerrero	Pícaro	-15%	+15% defensa efectiva del Pícaro
Tanque	Guerrero	-15%	+15% defensa efectiva del Guerrero
Pícaro	Tanque	-15%	+15% defensa efectiva del Tanque

ModTriangulo se calcula consultando esta tabla según el rol del atacante y el rol del objetivo. Los enfrentamientos neutrales (mismo rol) usan ModTriangulo = 1.0.

3.3 Golpe Crítico

Fórmula de Crítico

```
ProbabilidadCritico = CriticoBase_Atacante - EvasionCritico_Objetivo
```

```
Si Random(0,100) <= ProbabilidadCritico:
```

```
    ModCritico = 1 + (CriticoDaño_Atacante / 100)
```

```
Si no:
```

```
    ModCritico = 1.0
```

3.4 Orden de Turnos (Iniciativa)

Fórmula de Iniciativa

```
Iniciativa_Personaje = Velocidad_Personaje + Random(0, Velocidad_Personaje * 0.10)
```

El orden de actuación en cada ronda es descendente por Iniciativa.

Se recalcula cada ronda para permitir que buffs/debuffs de velocidad alteren el orden .

3.5 Efectos de Estado (Sangrado, Veneno, Aturdimiento)

Daño por Tick (Sangrado / Veneno)

```
DañoPorTick = VidaMaxima_Objetivo * PorcentajeEfecto
```

Sangrado típico: PorcentajeEfecto = 0.04 (4% de vida máxima por turno)

Veneno típico: PorcentajeEfecto = 0.03, ignora FactorMitigacion (daño verdadero)

DuracionTurnos: definida por la habilidad que aplica el efecto (típicamente 3-5 turnos)

3.6 Ruptura de Escudo

Fórmula de Ruptura

```
EscudoRestante -= DañoFinal * FactorRupturaTipo

FactorRupturaTipo = 1.5 si el tipo de daño coincide con la debilidad del escudo
FactorRupturaTipo = 0.5 si no coincide

Si EscudoRestante <= 0 dentro de VentanaRupturaSeg:

    Jefe entra en estado 'Aturdido' -> recibe ModCritico x2 durante N turnos

Si no se rompe a tiempo:

    Jefe ejecuta 'AtaqueDeCastigo' (alto daño en area)
```

3.7 Modo Automático vs. Intervención Manual

El modo automático ejecuta las habilidades según una lista de prioridad configurada por el jugador en la pantalla de build (ej.: 1. Curar si aliado <30% vida, 2. Definitiva si está disponible, 3. Ataque básico al objetivo con menos vida). El peso relativo se modela así:

Ponderación de Resultado

```
ResultadoCombate = (0.80 * CalidadDePreparacion) + (0.20 * CalidadDeIntervencionManual)

CalidadDePreparacion = f(nivel, talentos correctos, equipo correcto, plantilla adecuada)

CalidadDeIntervencionManual = f(interrupciones exitosas, combos sincronizados a tiempo)
```

Esta ponderación no debe programarse como una fórmula literal en el motor de combate — es una guía de diseño para calibrar cuánto deben pesar las interrupciones manuales (ej. cuánto daño extra otorga interrumpir la definitiva de un jefe) de modo que el resultado agregado se acerque a esta proporción en la mayoría de combates.

4. Módulo de Progresión de Personaje — Fórmulas

4.1 Experiencia y Nivel

Curva de Experiencia

```
XPRequeridaParaNivel(n) = 100 * (n ^ 1.5)
```

Ejemplos:

```
Nivel 2  -> 100 * 2^1.5    = 283 XP
```

```
Nivel 10 -> 100 * 10^1.5  = 3,162 XP
```

```
Nivel 30 -> 100 * 30^1.5  = 16,432 XP
```

El exponente 1.5 produce una curva de crecimiento moderado: ni plana (grind irrelevante) ni exponencial agresiva (pared de progreso). Ajustar el exponente entre 1.3 y 1.7 según ritmo deseado.

4.2 Crecimiento de Estadísticas

Estadística por Nivel

```
Estadistica(n) = EstadisticaBase + (CrecimientoPorNivel * (n - 1)) * MultiplicadorClase
```

MultiplicadorClase ejemplo:

```
Tanque  -> Vida x1.3 / Ataque x0.8
```

```
Picaro  -> Velocidad x1.2 / Critico x1.3
```

```
Guerrero-> Ataque x1.15 / Vida x1.0
```

4.3 Mastería por Uso

Ganancia y Tope de Mastería

```
GananciaMasteryPorCombate = 5 + (2 * TurnosActivoEnCombate)
```

```
MasteryActual = min(MasteryActual + Ganancia, TopeMasteryDelHito)
```

TopeMasteryPorHito:

Prologo -> Tope 20

Capitulo 1 -> Tope 45

Capitulo 2 -> Tope 75

Capitulo 3+ -> Tope 100 (maximo de la version 1.0)

Cada 10 puntos de maestría otorgan una mejora pequeña y permanente (ej. +1% crítico, o al llegar al tope narrativo, una ranura de pasiva secundaria). Los primeros 40 puntos deben sentirse notorios; los últimos 60, principalmente cosméticos o de refinamiento (retornos decrecientes deliberados).

4.4 Puntos de Talento por Nivel

Asignación de Puntos

```
PuntosTalentoGanados(n) = 1 punto por nivel
```

```
    + 1 punto adicional en niveles multiplos de 5 (hito menor)
```

```
    + 2 puntos adicionales en niveles multiplos de 10 (hito mayor
```

```
)
```

5. Módulo de Árbol de Talentos (Red por Bloques)

5.1 Costos por Tipo de Nodo

Tipo de Nodo	Costo (puntos)	Cantidad máx. por héroe	Efecto típico
Núcleo	3	2-3	Cambia comportamiento de IA o rol
Respuesta (Counter)	2	3-4	Contrarresta una mecánica/estado específico
Sincronización (Combo)	2	2-3	Sinergia condicional con otro rol/clase
Modificador (Stat)	1	Sin límite dentro del árbol	Ajuste fino porcentual

5.2 Sistema de Etiquetas (Tags) — el puente con el Diagnóstico

Cada nodo de tipo Respuesta debe declarar uno o más 'contrarresta_tags'. El motor de Diagnóstico (sección 6) genera exactamente esas mismas etiquetas al analizar una derrota. Esta correspondencia 1 a 1 es lo que permite sugerir nodos automáticamente sin lógica ad-hoc por cada combate.

Ejemplo de correspondencia

```
Diagnostico detecta  -> tag: 'sangrado_dominante'

Nodo 'Sangre Fria'  -> contrarresta_tags: ['sangrado_dominante', 'veneno_dominante'
]

Motor de sugerencia  -> resalta 'Sangre Fria' en el arbol del personaje afectado
```

5.3 Reseteo y Plantillas

Costo de Reseteo Nominal

```
CostoReseteo = 25 * NivelPersonaje (moneda: Oro general)
```

Reglas:

- Cambiar entre Plantillas guardadas (2-3 slots): SIEMPRE gratuito, fuera de combate.
- Reasignar puntos DENTRO de una plantilla ya guardada: cuesta CostoReseteo.
- Una vez iniciada la Mision Principal: el arbol queda bloqueado hasta el fin del intento.

6. Módulo de Diagnóstico de Derrota (Motor de Causalidad)

Este es el sistema más diferenciador del proyecto y el que requiere más cuidado de implementación. Debe registrar eventos durante el combate y, al perder, calcular qué causa contribuyó más a la derrota — no una lista genérica de eventos, sino una causa priorizada.

6.1 Registro de Eventos (Combat Log)

Estructura del evento

```
{  
  
  "turno": 14,  
  
  "tipo": "dano_recibido",      // dano_recibido | muerte | escudo_no_roto | curacion_  
insuficiente  
  
  "objetivo": "tanque_01",  
  
  "fuente_tag": "sangrado",    // etiqueta de la causa de daño  
  
  "cantidad": 340,  
  
  "vida_restante_pct": 12  
  
}
```

6.2 Algoritmo de Ranking de Causas

Pseudocódigo del análisis post-combate

```
funcion analizarDerrota(combatLog):  
  
    contribuciones = {}  
  
    para cada evento en combatLog:  
  
        si evento.tipo en ['dano_recibido', 'muerte']:  
  
            tag = evento.fuente_tag  
  
            contribuciones[tag] += evento.cantidad  
  
    total = suma(contribuciones.valores())  
  
    ranking = ordenar_descendente(contribuciones)  
  
    causaPrincipal = ranking[0]  
  
    porcentajeContribucion = (causaPrincipal.valor / total) * 100  
  
    retornar {  
  
        "causa": causaPrincipal.tag,  
  
        "contribucion_pct": porcentajeContribucion,  
  
        "mensaje": PLANTILLAS_MENSAJE[causaPrincipal.tag]  
  
    }
```

6.3 Umbral de Confianza

Regla de decisión

```
Si porcentajeContribucion >= 40%:  
  
    Mostrar UNA causa clara ('Tus tanques cayeron ante daño de sangrado')  
  
Si no hay causa por encima de 40%:  
  
    Mostrar mensaje generico de 'derrota por desgaste general'  
  
    y sugerir revisar nivel de personajes en vez de un contador especifico
```

Este umbral evita que el sistema invente una causa falsa cuando la derrota fue realmente producto de una combinación difusa de factores. Es preferible admitir ambigüedad que dar un diagnóstico incorrecto y erosionar la confianza del jugador en el sistema.

6.4 Motor de Sugerencia (conecta con el Árbol de Talentos)

Pseudocódigo de sugerencia

```
funcion sugerirSolucion(causa, personajesEnEquipo):  
  
    candidatos = []  
  
    para cada personaje en personajesEnEquipo:  
  
        para cada nodo en personaje.nodosDisponiblesNoDesbloqueados:  
  
            si causa.tag en nodo.contrarresta_tags:  
  
                candidatos.append({personaje, nodo})  
  
    si candidatos vacio:  
  
        buscar en catalogo de items/materiales con tag_contador == causa.tag  
  
        retornar sugerencia de mazmorra secundaria correspondiente  
  
    retornar candidatos    // se resaltan visualmente en la pantalla de derrota
```

7. Módulo de Expediciones y Energía

7.1 Regeneración de Energía

Fórmula de Regeneración

`EnergiaActual = min(EnergiaMaxima, EnergiaActual + 1 unidad cada 6 minutos)`

`EnergiaMaxima_base = 120 unidades/dia (aprox. 12 horas para regenerar desde 0)`

Bono pasivo (Idle): Oro_y_XP_basica se acumulan de forma continua e independiente de la Energia, con un tope de acumulacion de 8 horas sin conexion.

7.2 Costo de Mazmorras Secundarias

Tipo de Mazmorra	Costo de Energía	Recompensa
Materiales de Ascensión	15	Material específico de mejora de equipo
Tomos de Habilidad	20	Puntos de talento adicionales / libros de reseteo
Equipo Especializado	25	Piezas de equipo con tag_contador específico
Resistencias Elementales	15	Materiales para reducir daño de un elemento

Con 120 de energía diaria, el jugador puede completar entre 5 y 8 mazmorras secundarias por día, suficiente para atacar 1-2 problemas de diagnóstico distintos por sesión sin sentirse limitado ni sobrecargado.

8. Módulo de Reclutamiento Táctico

8.1 Moneda de Reputación (independiente del Oro)

Obtención y Costo

```
ReputacionGanada = 10 por Mision Principal completada  
  
                + 5 por Mazmorra secundaria (primera vez por semana)  
  
                + 25 por completar una Mision de Reclutamiento  
  
CostoContratoHeros = 150-400 Reputacion, segun relevancia narrativa del heroe
```

Al ser una moneda separada del Oro (usado para reseteo de talentos y mejoras de equipo), el jugador nunca enfrenta la disyuntiva de "reclutar un héroe nuevo" vs. "mejorar el equipo actual" — ambos progresan en paralelo.

8.2 Flujo de una Misión de Reclutamiento

- El tablón de contratos muestra el héroe objetivo desde el inicio (sin azar).
- Cadena de 2-4 misiones cortas que narran el trasfondo del héroe.
- Al completar la cadena, el héroe se une permanentemente al roster.
- El nivel inicial del héroe reclutado escala con el progreso narrativo del jugador (nunca se une un héroe subnivelado que quede obsoleto de inmediato).

9. Módulo de Economía y Monetización

9.1 Monedas del Sistema

Moneda	Obtención	Uso
Oro	Combates, Idle, mazmorras	Reseteo de talentos, mejora de equipo
Reputación	Misiones, mazmorras (semanal)	Contratos de reclutamiento
Materiales de Ascensión	Mazmorras específicas	Subir el tope de equipo/rareza de objetos
Gemas Cosméticas (premium opcional)	Compra directa	Skins, temas de UI, slots de inventario

9.2 Pase de Batalla Sin Caducidad

Lógica de progreso del pase

```
PuntosDePase += actividad_normal_del_juego (mision, mazmorra, mastery)
```

```
NivelDePase = floor(PuntosDePase / 500)
```

Recompensas: exclusivamente cosmeticas o de conveniencia

(slots de plantilla extra, skins, marcos de perfil, temas de interfaz).

El pase NUNCA expira ni resetea progreso ya ganado.

10. Arquitectura de Software Recomendada

Se recomienda una separación estricta entre lógica de simulación (sin estado visual) y presentación, de modo que el balance pueda testearse mediante scripts automatizados (miles de combates simulados) sin necesidad de ejecutar el juego completo.

10.1 Módulos Independientes

Módulo	Responsabilidad	Dependencias
CombatResolver	Cálculo de daño, turnos, estados, ruptura de escudo	CharacterData, EncounterData
ProgressionService	XP, nivel, maestría, puntos de talento	CharacterData
TalentTreeManager	Validación de nodos, costos, plantillas, reseteo	CharacterData, TalentNodeDB
DiagnosticEngine	Registro de combate y ranking de causas	CombatResolver (log de eventos)
SuggestionEngine	Cruce de tags entre diagnóstico y nodos/items	DiagnosticEngine, TalentTreeManager, ItemDB
ExpeditionService	Energía, costos, recompensas de mazmorras	PlayerEconomy
RecruitmentService	Reputación, contratos, desbloqueo de héroes	PlayerEconomy, NarrativeFlags
EconomyService	Monedas, pase de batalla, tienda cosmética	PlayerData
UI Layer	Pantallas de build, derrota, expedición, combate	Todos los anteriores (solo lectura)
SaveSystem	Persistencia de todo el estado del jugador	Todos los módulos

■ *Regla de arquitectura: ningún módulo de lógica debe importar código de la capa de UI. Esto permite escribir tests automatizados que simulan miles de combates para detectar builds rotas o desequilibradas antes de que lleguen a un jugador real.*

11. Plan de Construcción del Prototipo (Sprints)

Plan orientativo de 6 sprints de 2 semanas (12 semanas / 3 meses) para llegar a un prototipo jugable de principio a fin del bucle central, sin arte final.

Sprint	Objetivo	Entregable verificable
1	CombatResolver + esquemas de datos	Combate simulado por consola/log, sin UI, con las 3 clases base
2	ProgressionService + TalentTreeManager	Personajes suben de nivel y asignan nodos desde una interfaz mínima
3	UI de Combate + Modo Automático	Un combate completo jugable de principio a fin contra el jefe de prueba
4	DiagnosticEngine + SuggestionEngine	Pantalla de derrota funcional con causa priorizada y nodo sugerido
5	ExpeditionService + 2 mazmorras secundarias	El jugador puede farmear el contador exacto sugerido por el diagnóstico
6	Integración, balance y pulido del vertical slice	Loop completo jugable: derrota -> diagnóstico -> mazmorra -> victoria

12. Métricas de Balance a Vigilar

Durante las pruebas del prototipo, instrumentar el juego para registrar estas métricas permite detectar desequilibrios antes de que se conviertan en problemas de diseño arraigados.

Métrica	Qué revela	Señal de alerta
% de victorias en 1er intento vs. 3er intento	Si el diagnóstico realmente ayuda a progresar	Si no mejora entre intentos, el diagnóstico o la sugerencia fallan
Distribución de nodos elegidos por jugadores	Si el árbol converge en una única build 'meta'	Concentración >70% en los mismos nodos entre distintos jugadores
Tiempo promedio entre derrota y reintento exitoso	Ritmo real de sesión corta vs. idle largo	Si supera varias sesiones, la mazmorra objetivo da muy poco recurso
Uso relativo de las 3 clases base	Si el Triángulo Táctico está balanceado	Una clase usada <15% del tiempo en equipos ganadores
Ratio automático/manual en victorias	Si la ponderación 80/20 se sostiene en la práctica	Victorias 100% automáticas sugieren que el modo manual es prescindible

Con estos siete módulos funcionando de forma independiente y comunicándose únicamente a través de las estructuras de datos definidas en la sección 2, el equipo de desarrollo puede construir, testear y balancear el prototipo de forma incremental sin bloquear un sistema por otro — que es, en esencia, la misma filosofía de preparación consciente que el juego le propone al jugador, aplicada ahora al proceso de desarrollo.